

GUÍA DEL EJERCICIO PRÁCTICO
CRUD completo de estudiantes en Java con MySQL

Tema: 8.4 - CRUD completo en Java con MySQL

Estudiante: Karolayn Paola Buñay Pérez

Grupo 1: Java, JDBC y ORM

Fecha: 15 de julio de 2026

1. Introducción

Esta guía explica el ejercicio práctico desarrollado para demostrar un CRUD de estudiantes en Java conectado a MySQL. El programa permite registrar, consultar, buscar, actualizar y eliminar estudiantes. Se describe el mismo ejercicio presentado en el código, incluyendo su configuración, ejecución y verificación.

2. Objetivo

Implementar un sistema básico de gestión de estudiantes mediante Java, JDBC y MySQL, aplicando las operaciones Create, Read, Update y Delete sobre información almacenada permanentemente.

3. Herramientas y requisitos

El ejercicio utiliza Java, Maven, JDBC, MySQL y MySQL Connector/J. Puede ejecutarse desde Visual Studio Code, IntelliJ IDEA o NetBeans.

1. Tener instalado Java JDK 17 o una versión compatible.
2. Tener Maven disponible.
3. Mantener activo el servicio MySQL.
4. Crear la base de datos universidad_java y la tabla estudiantes.
5. Revisar el usuario y la contraseña configurados en Java.

4. Creación de la base de datos

Después de iniciar MySQL desde XAMPP, MySQL Workbench o el servicio instalado en el equipo, se ejecuta el siguiente script:

```
CREATE DATABASE IF NOT EXISTS universidad_java;  
USE universidad_java;
```

```
CREATE TABLE IF NOT EXISTS estudiantes  
( id INT AUTO_INCREMENT PRIMARY  
  KEY, nombre VARCHAR(100) NOT NULL,  
  carrera VARCHAR(100) NOT NULL,  
  semestre INT NOT NULL  
);
```

El campo id identifica de forma única a cada estudiante y se genera automáticamente. Los campos nombre, carrera y semestre almacenan la información académica del registro.

5. Organización del proyecto

ConexionBD.java: abre la conexión con MySQL.

Estudiante.java: representa los datos de un estudiante.

EstudianteDAO.java: contiene las operaciones de insertar, consultar, buscar, actualizar y eliminar.

Main.java: ejecuta la demostración del CRUD.

pom.xml: incluye la dependencia de MySQL Connector/J.

6. Configuración de la conexión

La clase ConexionBD.java contiene la dirección de la base de datos, el usuario y la contraseña. En una instalación local con XAMPP normalmente se usa el usuario root y una contraseña vacía.

```
private static final String URL =  
    "jdbc:mysql://localhost:3306/universidad_java";  
private static final String USUARIO = "root";  
private static final String CONTRASENA = "";  
  
public static Connection obtenerConexion() throws SQLException {  
    return DriverManager.getConnection(URL, USUARIO, CONTRASENA);  
}
```

localhost indica que MySQL se encuentra en el mismo equipo, 3306 es el puerto habitual y universidad_java es la base de datos utilizada.

7. Operaciones realizadas

Las consultas se ejecutan con PreparedStatement. Los signos de interrogación representan los valores que Java asigna antes de ejecutar cada instrucción.

7.1 Registrar un estudiante

```
INSERT INTO estudiantes (nombre, carrera, semestre)  
VALUES (?, ?, ?);
```

El programa asigna los datos mediante setString() y setInt(). Después usa executeUpdate(), que devuelve la cantidad de filas insertadas.

7.2 Consultar estudiantes

```
SELECT id, nombre, carrera, semestre  
FROM estudiantes;
```

La consulta se ejecuta con executeQuery(). Los resultados se guardan en un ResultSet y se recorren con while (rs.next()).

7.3 Buscar por nombre

```
SELECT id, nombre, carrera, semestre  
FROM estudiantes  
WHERE nombre LIKE ?;
```

El símbolo de porcentaje permite búsquedas parciales. Por ejemplo, %Karo% encuentra el registro Karolayn.

7.4 Actualizar un estudiante

```
UPDATE estudiantes  
SET carrera = ?, semestre = ?  
WHERE id = ?;
```

La condición WHERE modifica únicamente el registro seleccionado. En la prueba, Karolayn pasa de Software, semestre 2, a Ingeniería de Software, semestre 3.

7.5 Eliminar un estudiante

```
DELETE FROM estudiantes  
WHERE id = ?;
```

El registro se elimina mediante su identificador. La condición WHERE evita que se borren todos los estudiantes de la tabla.

8. Ejecución del ejercicio

Desde Main.java se realizan las siguientes acciones:

1. Registrar a Karolayn, Eduardo y Emilia.
2. Mostrar todos los estudiantes.
3. Buscar a Karolayn mediante una parte de su nombre.
4. Actualizar la carrera y el semestre de Karolayn.
5. Eliminar el registro de Eduardo.
6. Consultar nuevamente la tabla.

Las llamadas utilizadas en la demostración son:

```
insertarEstudiante("Karolayn", "Software", 2);  
insertarEstudiante("Eduardo", "Software", 2);  
insertarEstudiante("Emilia", "Diseño Gráfico", 4);  
  
listarEstudiantes();  
buscarEstudiantePorNombre("Karo");  
actualizarEstudiante(1, "Ingeniería de Software", 3);  
eliminarEstudiante(2);  
listarEstudiantes();
```

9. Cómo ejecutar y verificar

1. Iniciar MySQL.
2. Ejecutar el archivo universidad_java.sql.
3. Abrir el proyecto Maven.
4. Revisar los datos de conexión.
5. Ejecutar Main.java.
6. Observar los mensajes de la consola.
7. Consultar la tabla estudiantes en MySQL.

Al finalizar, los registros principales deben quedar así:

ID 1: Karolayn - Ingeniería de Software - semestre 3.

ID 3: Emilia - Diseño Gráfico - semestre 4.

Eduardo ya no aparece porque su registro fue eliminado durante la demostración.

10. Buenas prácticas aplicadas

Se utilizaron PreparedStatement, try-with-resources, validación de datos y manejo de SQLException. Además, se verificó la cantidad de filas afectadas después de insertar, actualizar o eliminar.

11. Errores frecuentes

Los errores más comunes son trabajar con MySQL apagado, usar credenciales incorrectas, olvidar MySQL Connector, indicar un ID inexistente o repetir las inserciones iniciales.

12. Conclusión

El ejercicio demuestra el funcionamiento de un CRUD en Java con MySQL. JDBC permite la comunicación con la base de datos, PreparedStatement ejecuta consultas seguras y ResultSet permite leer los registros. Estos conocimientos pueden aplicarse en sistemas académicos, inventarios y otras aplicaciones.